

Infraestrutura Operacional de IA

Como o IOAID Sustenta a Operação Diária da Rede Nexus

MMN_IA Collective

2026-06-12

Capa — Infraestrutura Operacional de IA

Capa — Infraestrutura Operacional de IA

Infraestrutura Operacional de IA

Como o IOAID Sustenta a Operação Diária da Rede Nexus

Por MMN_IA Collective · Academ'IA

Nexus Affil'IA'te · 2026

Sobre este documento

A maioria dos afiliados nunca vai precisar abrir um terminal, editar um arquivo YAML, ou rodar um comando `docker compose`. Mas todo afiliado deveria entender **o que acontece por trás do botão “Enviar”** que ele aperta todo dia. Este documento entrega exatamente isso: uma explicação arquitetural do IOAID sem jargão desnecessário, com diagramas, exemplos práticos, e o vocabulário que você precisa para conversar com o time técnico quando algo dá

errado. Se você é afiliado, vai sair daqui entendendo a máquina. Se é engenheiro, vai encontrar o mapa que normalmente só se obtém em 3 meses de código.

Sumário

- 1. O que é IOAID em uma frase
- 2. Os 5 módulos explicados com analogia
- 3. O fluxo completo de um disparo de WhatsApp
- 4. Isolamento e sandbox: por que uma falha não derruba tudo
- 5. Custos operacionais: quanto custa cada ação
- 6. Latência e SLA: o que esperar e quando reclamar
- 7. Telemetria: o que está sendo medido em você
- 8. Resiliência: como o sistema se recupera
- 9. Limites conhecidos do IOAID em 2026
- 10. Como pedir suporte quando algo quebra

1. O que é IOAID em uma frase

IOAID é o sistema que pega uma intenção humana (“quero disparar mensagem de Natal para meus 800 clientes frios”) e a executa de forma auditável, escalável, e reversível.

Três palavras-chave: **auditável** (você sabe o que foi feito), **escalável** (funciona com 10 ou 10.000 contatos), **reversível** (você pode desfazer ações problemáticas).

2. Os 5 módulos explicados com analogia

Pense no IOAID como um **restaurante de luxo**:

- **M1 — Ingestion** é o **maître**. Recebe o cliente, anota o pedido, passa para a cozinha.
- **M2 — Routing** é o **chef de partida**. Decide qual estação da cozinha (skill) prepara cada item.

- **M3 — Execution** é a **estação de cozinha**. Executa o preparo (skill) com utensílios próprios (sandbox).
- **M4 — Persistence** é o **livro de registro**. Anota tudo: hora, quem pediu, quanto custou, qualidade.
- **M5 — Response** é o **garçom**. Devolve o prato pronto (resultado) ao cliente, com recibo (metadados).

Cada módulo é independente. Se o livro de registro (M4) falhar, o restaurante continua servindo (M1, M2, M3, M5 funcionam), só não conseguimos cobrar direito. Se a estação de sobremesa (uma skill) falhar, o resto da cozinha segue.

3. O fluxo completo de um disparo de WhatsApp

Vamos acompanhar um disparo real, do clique ao envio.

Passo 1 — Você clica “Disparar” no painel. O painel envia um POST para `/api/v1/dispatch` com os parâmetros (mensagem, base, segmentação).

Passo 2 — M1 (Ingestion) recebe. Valida o token de autenticação, valida os parâmetros, registra timestamp e usuário. Devolve um `request_id` único (ex: `req_a8f3e2c1`).

Passo 3 — M2 (Routing) decide. Olha o `skill-manifest.json` da skill `whatsapp-dispatch` e descobre: precisa das skills `audience-segmenter` + `copy-personalizer` + `whatsapp-sender`. Encadeia a execução.

Passo 4 — M3 (Execution) executa cada skill em sandbox. - `audience-segmenter` filtra os 800 contatos da coorte “frios” → retorna 800 IDs. - `copy-personalizer` personaliza a mensagem para cada contato (nome, último produto) → 800 mensagens. - `whatsapp-sender` enfileira as 800 mensagens com rate-limit de 80/segundo.

Passo 5 — Judge Revisor avalia cada mensagem. O

Judge olha as 800 mensagens. Se detectar spam, bloqueia e marca para revisão. Se aprovar, libera.

Passo 6 — M4 (Persistence) registra. Cada envio vira uma

linha no log: `request_id`, `contato_id`, `mensagem_hash`, `timestamp`, `custo_tokens`, `aprovado_por_judge`.

Passo 7 — M5 (Response) devolve. Você recebe no painel:

“Disparadas 798 de 800 mensagens. 2 bloqueadas pelo Judge (spam detectado). Revise e aprove.”

Tempo total: ~14 segundos para 800 mensagens.

4. Isolamento e sandbox: por que uma falha não derruba tudo

Cada execução de skill roda em **sandbox isolado**. Isso significa:

- Uma skill que consome toda a memória é morta antes de derrubar o sistema.
- Uma skill com loop infinito é morta após timeout (default: 30s).
- Uma skill que tenta acessar filesystem é bloqueada por padrão.

O SHO monitora todos os sandboxes. Se um começa a falhar repetidamente, **isola** e migra tráfego para o fallback.

5. Custos operacionais: quanto custa cada ação

Cada ação tem custo em **tokens** (do LLM) e em **infra** (CPU, memória, rede). Os custos típicos em 2026:

- **Disparo WhatsApp** (1 mensagem com personalização): ~120 tokens ≈ R\$ 0,0024
- **Análise de coorte** (1 query): ~800 tokens ≈ R\$ 0,016
- **Geração de copy** (1 peça): ~600 tokens ≈ R\$ 0,012

- **Judge Revisão** (1 mensagem): ~200 tokens ≈ R\$ 0,004
- **Operação completa** (1 disparo de 800 mensagens):
~1.000.000 tokens ≈ R\$ 20

O custo médio por afiliado ativo: **R\$ 80-200/mês**, dependendo do volume. Comparado a 1 atendente humano (R\$ 1.800/mês), a economia é brutal.

6. Latência e SLA: o que esperar e quando reclamar

O SLA do IOAID em 2026 é de **< 2 segundos para 95% das ações**.

Ações que costumam respeitar o SLA: - Disparos em massa (até 1.000 mensagens): 5-15s total. - Análises de corte: 1-3s. - Geração de copy: 2-5s. - Judge Revisão (1 mensagem): 200-500ms.

Ações que **podem estourar** o SLA: - Disparo massivo (10.000+ mensagens): pode levar 2-3 min. - Lançamento simultâneo de 5+ nós federados: latência de rede. - Cold start de skill nova: primeira execução é mais lenta.

Se você está vendo latência > 10s consistentemente, **abrir issue de suporte** (vide seção 10).

7. Telemetria: o que está sendo medido em você

O IOAID mede **tudo**, e os dados são **seus** (você vê no dashboard):

- **Latência por skill** (p50, p95, p99).
- **Custo em tokens/USD por ação**.
- **Taxa de aprovação do Judge**.
- **Taxa de erro por skill**.
- **Uso de sandbox** (% de execuções que precisaram de retry).
- **Consumo de quota** (vs limite do seu plano).

Esses dados são **auditados por terceiro** trimestralmente (relatório em `Lib-Nexus/audits/`).

8. Resiliência: como o sistema se recupera

Quando uma falha acontece, o sistema segue uma **árvore de decisão**:

1. **Retry com backoff**: erro de rede? Tenta de novo em 1s, 2s, 4s. Sucesso em 80% dos casos.
2. **Fallback para skill alternativa**: skill primária falhou 3x? Troca para skill secundária (configurável).
3. **Modo degradação**: skill crítica caiu? IOAID entra em modo leitura (você vê, mas não envia) e alerta humano.
4. **Quarentena + alerta**: falha grave. SHO bloqueia skill e abre ticket.

Em 99,2% dos incidentes, **você nem percebeu que houve falha**. O sistema se recuperou sozinho.

9. Limites conhecidos do IOAID em 2026

- **Cold start de skill nova**: primeira execução é 3-5x mais lenta.
- **Limite de soft de 100 skills por agente**: ultrapassar = degradação de performance.
- **Rate limit do WhatsApp**: o que limita mesmo é a Meta, não o IOAID.
- **Cold start do Judge**: se o Judge ficar offline, mensagens entram em fila (não são enviadas sem aprovação).
- **Custo de cold start**: subir nó novo na federação custa ~R\$ 5 de provisionamento.

10. Como pedir suporte quando algo quebra

Quando algo dá errado:

1. **Copie o** `request_id` da operação que falhou (aparece no painel).
2. **Abra ticket** em `https://github.com/Nexus-HUB57/MMN_AI-to-AI/issues/new` com label `suporte-ioaid`.
3. **Cole:** `request_id`, hora exata, ação que tentou executar, mensagem de erro.
4. **Aguarde SLA:** resposta inicial em 2h úteis, resolução em 24h para incidente, 72h para enhancement.

Tenha em mãos também: plano do seu nó, sua coorte de teste (se aplicável), e o print do erro. Quanto mais contexto, mais rápido.

Infraestrutura Operacional de IA — Por MMN_IA Collective

MMN AI-to-AI · 2026 · Todos os direitos reservados